PRODUCT / ENGINEERING / OPERATIONS

Matchora

Professional documentation

A comprehensive guide to the product, architecture, APIs, data, security and production operations.

EVIDENCE-LED EDITION

91 text files / 8 models / 20 API operations

Source verified • Live public screens • Explicit limitations



8 NOVEMBER 2026 / EDITION 1.0

`main @ d72d00380934bf62b9e0e31eeaa24023ec461032`

github.com/abhinav84094/Matchora

matchora.co.in

READ THIS FIRST

A precise implementation snapshot

Professional documentation grounded in code and public-page observations.

Document control	Value
Edition / review date	1.0 / 8 November 2026 (Asia/Kolkata)
Repository / branch	abhinav84094/Matchora / main
Pinned revision	d72d00380934bf62b9e0e31eeaa24023ec461032
Commit authored	22 October 2026, 07:23 UTC - SEO metadata configured
Evidence coverage	97 tracked blobs: 91 text files inspected; 6 binary/vector image assets inventoried. 43 backend JavaScript syntax checks and 7 isolated helper checks.
Review method	Read-only GitHub retrieval; public browser inspection; isolated local checks. No repository writes, commits, live uploads, payments, or database changes.

How to interpret the evidence

Label	Meaning
CODE VERIFIED	Directly supported by the pinned implementation. This does not prove the feature succeeds in production.
LIVE OBSERVED	Visible on the public website during this review. Observations are time-bound.
DISABLED / INCOMPLETE	Present in files but commented out, unmounted, stubbed, or unfinished.
NOT DOCUMENTED / UNVERIFIED	The inspected sources do not establish the detail, or access was unavailable.
RECOMMENDED	Proposed work or operating practice; not a claim about the current system.

Code takes precedence over README prose and inline comments. Examples use synthetic identifiers and are illustrative response shapes, not captured production records. Source IDs link to immutable GitHub files; the source register explains each ID.

Scope boundary

No authenticated end-to-end session, production configuration, Search Console account, database contents, provider dashboards, or hosting logs were available. This is implementation documentation with an evidence-based review, not a penetration test, compliance certification, or uptime report.

NAVIGATION

Find what you need

A reading guide for product users, maintainers, and reviewers.

Pages	Section / purpose
4-9	Product and user manual - capabilities, live screens, resume upload, job discovery, applications, billing and administration.
10-15	Architecture and processing - system boundaries, frontend, resume analysis, recommendations, ATS scoring and job lifecycle.
16-19	Data design - entity relationships, fields, indexes, persistence rules and dormant structures.
20-28	API reference - all 20 mounted API operations, inputs, outputs, error behavior and billing contracts.
29-32	Configuration and operations - environment variables, local setup, deployment, maintenance and troubleshooting.
33-34	Security and reliability - implemented controls, gaps and prioritized engineering findings.
35-36	SEO and testing - verified metadata, crawl uncertainty, actual checks and release acceptance criteria.
37	Limitations and roadmap - current boundaries and proposed phases.
38-40	Evidence register and coverage - source links, review inventory and terminology.

Suggested reading paths

New user: start at pages 4-8. The user manual describes code-supported behavior; authenticated screens are explained from source rather than represented as live screenshots.

New developer: read pages 10-19, then environment setup and API details. Pay particular attention to candidate limits, date parsing, plan expiry and disabled jobs.

Operator or owner: read pages 9, 28-36. Verify deployed configuration and recovery paths before relying on production billing, data retention or job freshness.

> **Visual integrity**
> Screenshots show actual public pages. Technical diagrams are authored from code and labeled with their evidence. They depict logical relationships, not a verified physical hosting topology. No traffic, revenue, user-count or performance charts have been fabricated.

Version discipline

This document is pinned to the revision on page 2. Recheck the branch head, routes, schemas, plan constants and public pages when publishing a new edition. A website can be deployed from a different revision; deployment parity was not established.

01 / PRODUCT

From resume to relevant opportunities

Matchora is a candidate-first job matching and application-recording web application.

The product helps students, graduates and software professionals understand a resume, discover relevant stored job listings and keep a record of applications. Google sign-in identifies the candidate. Gemini structures resume text; deterministic matching then compares skills and experience with MongoDB job records.

Capability	Verified implementation / boundary
Identity	Google ID-token verification; seven-day JWT cookie; protected user and admin APIs.
Resume intelligence	Text-based PDF extraction, structured AI analysis, canonical skills, strengths, gaps, suggestions and a custom ATS score.
Recommendations	Active stored jobs, skill overlap and experience eligibility. Location filtering; Pro pagination and platform selection. No live scraping occurs per request.
Applications	User-confirmed creation and listing. New records are Applied. Status editing and deletion are not mounted routes.
Pro access	Razorpay order creation and signature verification; fixed 30-day / 365-day entitlement extension. No recurring subscription implementation found.
Administration	Admin email allowlist; KPIs, growth, job analytics, feedback statistics and permanent job deletion.
Study support	Protected Pro API uses static database resources. Frontend route is absent and seed call is disabled.
Installable web app	PWA manifest, service worker registration and conditional install prompt. Offline authenticated workflows were not verified.

What the product does not yet establish

The repository does not supply an operating job-ingestion worker, automatic job application system, account deletion API, application stage transitions, recurring billing, provider webhook reconciliation, production deployment manifests or a substantive active privacy policy.

> **Marketing versus implementation**
> The About page describes daily job updates and tracking from Applied to Offer, while the current source does not implement the ingestion schedule or status-update route. Treat these as product intent rather than verified behavior.

01 / LIVE WEBSITE

The public product experience

Live observation - https://matchora.co.in/ - 8 November 2026.

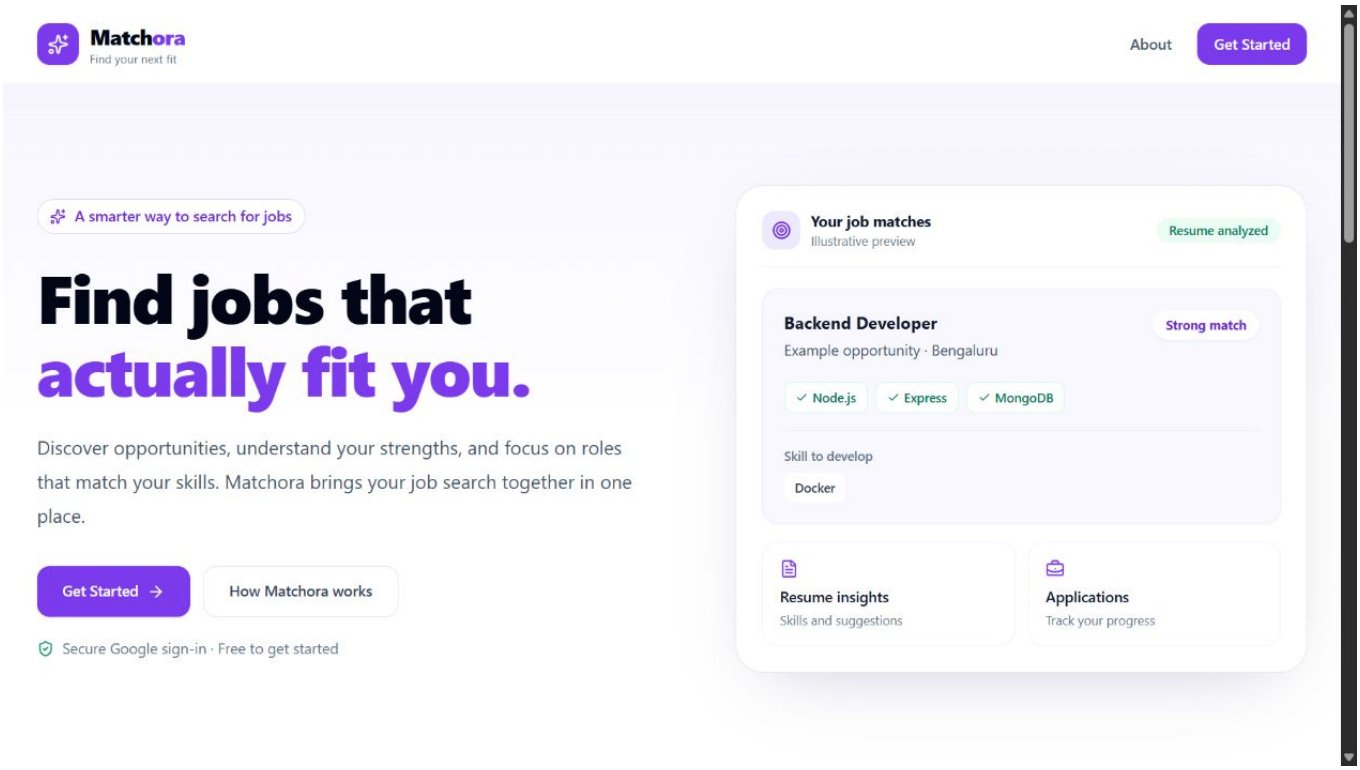


Figure 1. Actual homepage capture. The match card is explicitly an illustrative preview; it is not a production recommendation or a measured outcome.

What was visible

The homepage presents a resume-led workflow: sign in, add a resume and review matches. About and Privacy pages are linked, with Pricing in the footer. Google sign-in is offered, and the public message is free to get started.

Public observation	Interpretation
Home / About / Pricing rendered	Public marketing routes were accessible in the browser.
About focuses on LinkedIn and software roles	Matches the Free recommendation restriction; Pro supports additional platform values in code.
About says free for all users	Conflicts with the active Pro pricing and payment implementation.
Protected dashboard showed Loading...	No authenticated dashboard outcome was established during the observation. This alone does not establish a site-wide outage.

Review access

No sign-in, resume upload, application submission, checkout, admin access or production data retrieval was performed. The following manual describes source-verified flows.

EVIDENCE L01 home; L02 About; L03 Pricing; R30, R60

02 / USER MANUAL

Start with your resume

Source-verified instructions for account access and resume analysis.

1. Sign in with Google

Open the homepage and use Get Started to reach the sign-in area. Choose the Google sign-in control. The frontend submits the returned credential to POST /api/auth/google and navigates to /dashboard on success. Matchora uses a cookie session; protected pages redirect unauthenticated users to the homepage after the auth check completes.

2. Prepare and upload a readable PDF

Open Resume from the sidebar or use Upload Your Resume on the dashboard. Drag and drop a file or select Browse Files. The interface checks the .pdf extension and a maximum size of 5 MB. Use a text-based PDF; scans that yield fewer than 30 readable characters are rejected by the backend. OCR is not implemented.

3. Review the analysis

After success, review the ATS score, extracted skills, education, experience, projects, strengths, missing skills and improvement suggestions. The score is a custom heuristic, not a prediction of hiring success or certification by a third-party ATS. Review extracted details for errors before acting on recommendations.

Plan / state	Actual upload behavior
Free / no existing resume	One successful resume can be stored. Failed attempts do not create the entitlement lock unless a resume was persisted.
Free / resume already exists	Reupload returns 403. No user-facing delete or reset endpoint was found.
Active Pro	Existing resume can be replaced; each successful Pro upload sets nextUploadAt to 24 hours later.
Any upload attempt	Upload route has 10 attempts per hour per limiter key, plus the general API limit.
Temporary file	Stored on backend disk during processing; cleanup is attempted in finally on success, rejection or failure. Raw extracted text remains in MongoDB.

Troubleshooting

422 unreadable PDF: export a text-based PDF from your editor. **429:** use nextUploadAt or wait for the rate-limit window. **403 after a previous upload:** the Free entitlement is exhausted. **500:** retry after checking service availability; a malformed PDF, AI failure or database failure can share this response.

> **Upload limit caveat**
> The 5 MB and PDF extension checks are in the browser. Multer does not enforce size, type or file-signature restrictions on the server. This is a security gap, not a supported way to upload larger files.

02 / USER MANUAL

Discover, inspect and record a job

Applications are recorded after the user confirms applying on the hiring site.

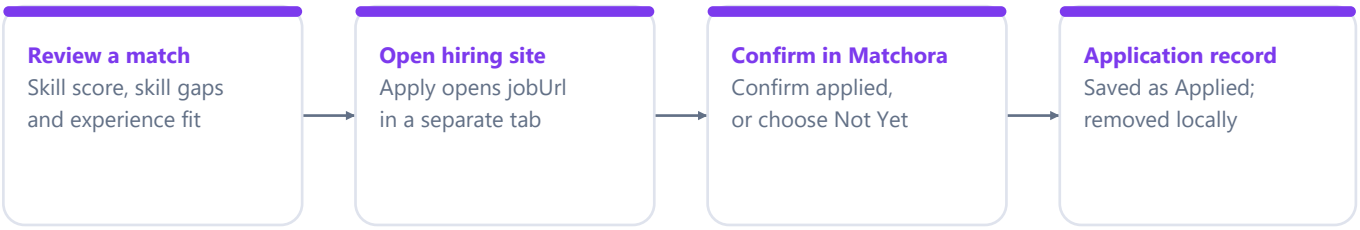


Figure 2. Source-derived application flow. No automatic submission to an employer is implemented.

Find jobs that match

Open Jobs For You. A resume and at least one inferred preferred role are needed for the current screen to initiate its fetch. Search by location and submit Search; location matching is a case-insensitive substring filter. Pro users can switch platform chips and load additional 25-job batches. Free requests are forced to LinkedIn and page 1 with a five-job response limit.

Understand the score and analysis

Open a job card analysis to inspect matched and missing skills, description and experience comparison. The displayed match percentage is required-skill overlap. Experience is an eligibility gate, not a blended score. Jobs below the configured skill threshold or required experience are omitted.

Apply and confirm

Apply opens the original listing with noopener/noreferrer. Finish the application on that external site, then confirm in Matchora. Only the confirmation posts an application record. Not Yet dismisses the confirmation without creating a record. A stored Applied record is a user report, not employer confirmation.

View application history

Open the sidebar application page (currently labeled Appied Jobs). Records show role, company, location, source, score and original listing. The model allows Saved, Interview, Offer and Rejected, but the current interface exposes only an Applied tab and no stage-editing control.

> **Recommendation limits**
> Counts are computed from a bounded candidate pool and include jobs the browser may hide as already applied. Free responses can expose hasNextPage even though the controller always forces page 1. A disappearing list does not mean there are no jobs in the database.

02 / USER MANUAL

Plans, profile and support

Public pricing is visible; successful payment activation was not exercised.

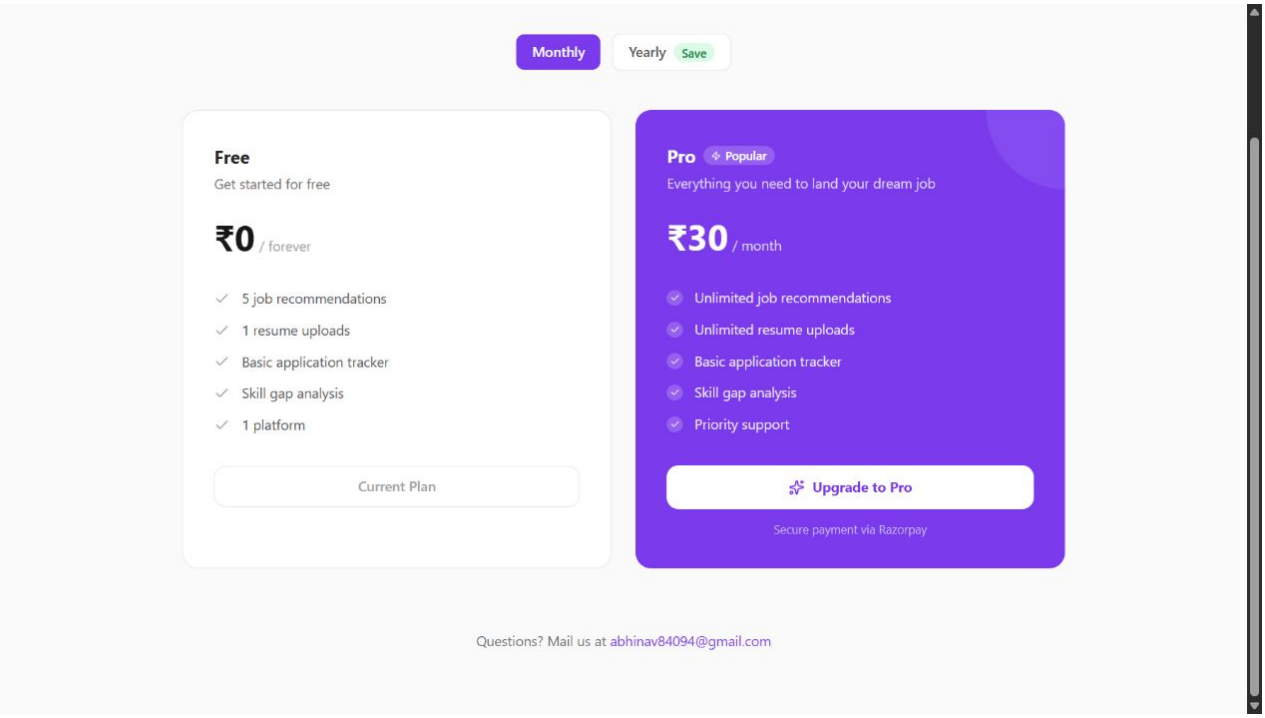


Figure 3. Actual monthly pricing page. Public prices are INR 30/month and INR 299/year on the review date. These are display values, not verified configured charge amounts.

Task	User action / current boundary
Upgrade	Open Pricing, choose Monthly or Yearly, then Upgrade to Pro after signing in. Razorpay checkout follows a server-created order.
Activation	The checkout callback posts verification details. Success reloads recommendations. Active entitlement is normally plan=pro with a future planExpiresAt.
Expiry / renewal	Purchases extend remaining active time by 30 or 365 days. No auto-renewal, cancellation or refund API was found.
Profile	View identity, resume details and plan expiry. Subscription progress uses account createdAt as a fallback; it is not a reliable purchase-start timeline.
Support / feedback	Dashboard supports a 13-step feedback survey. Email and WhatsApp links appear when configured. Pricing contact text and mailto target disagree.
Install	An Install button appears only when the browser offers an install prompt. Availability depends on the browser and current install state.

Plan wording

Unlimited recommendations means paged access to a bounded pool of at most 500 candidates per query. Unlimited resume uploads still carry a 24-hour cooldown. Server prices are configured separately from the hardcoded pricing display.

03 / ADMINISTRATION

Operate the admin workspace

Backend authorization uses an environment-based email allowlist.

Access and navigation

Sign in using an email configured in ADMIN_EMAILS. Open /admin directly. The frontend checks GET /api/admin/verify before rendering the workspace; the backend independently applies authMiddleware and adminMiddleware to every admin route. A hidden link or frontend guard is not the authorization boundary.

Panel / metric	Definition in the current controller
Total users	Count of all User documents.
Active users (30d)	Users whose lastLoginAt falls within the last 30 days; a sign-in measure, not general activity or unique daily use.
Total jobs KPI	Count of Job documents with status active.
Applications / resumes	Counts of all Application / Resume documents.
Growth	User, Application and Job creation counts grouped by UTC day; requested window defaults to 30 and clamps to 7-90 days.
Platform / top companies	Groups all jobs, including expired ones. Top company aggregation returns at most 10 groups.
Jobs today	Created in the last rolling 24 hours, not since local midnight.
Feedback	Total and New counts, rating distribution, average rating and average npsScore. avgNps is an average response score, not the conventional NPS formula.

Review and delete jobs

Open /admin/jobs. The browser loads all active jobs, then filters title/company and platform locally. Open a job for review and use the deletion confirmation only when removal is intended. DELETE /api/admin/jobs/:id permanently deletes the Job document. Application documents retain copied metadata and are not cascaded.

> **Operational caution**
> Job deletion has no soft-delete, restore route or audit log in this repository. The API does not paginate the admin list. Confirm the target and retain an appropriate backup before operational use.

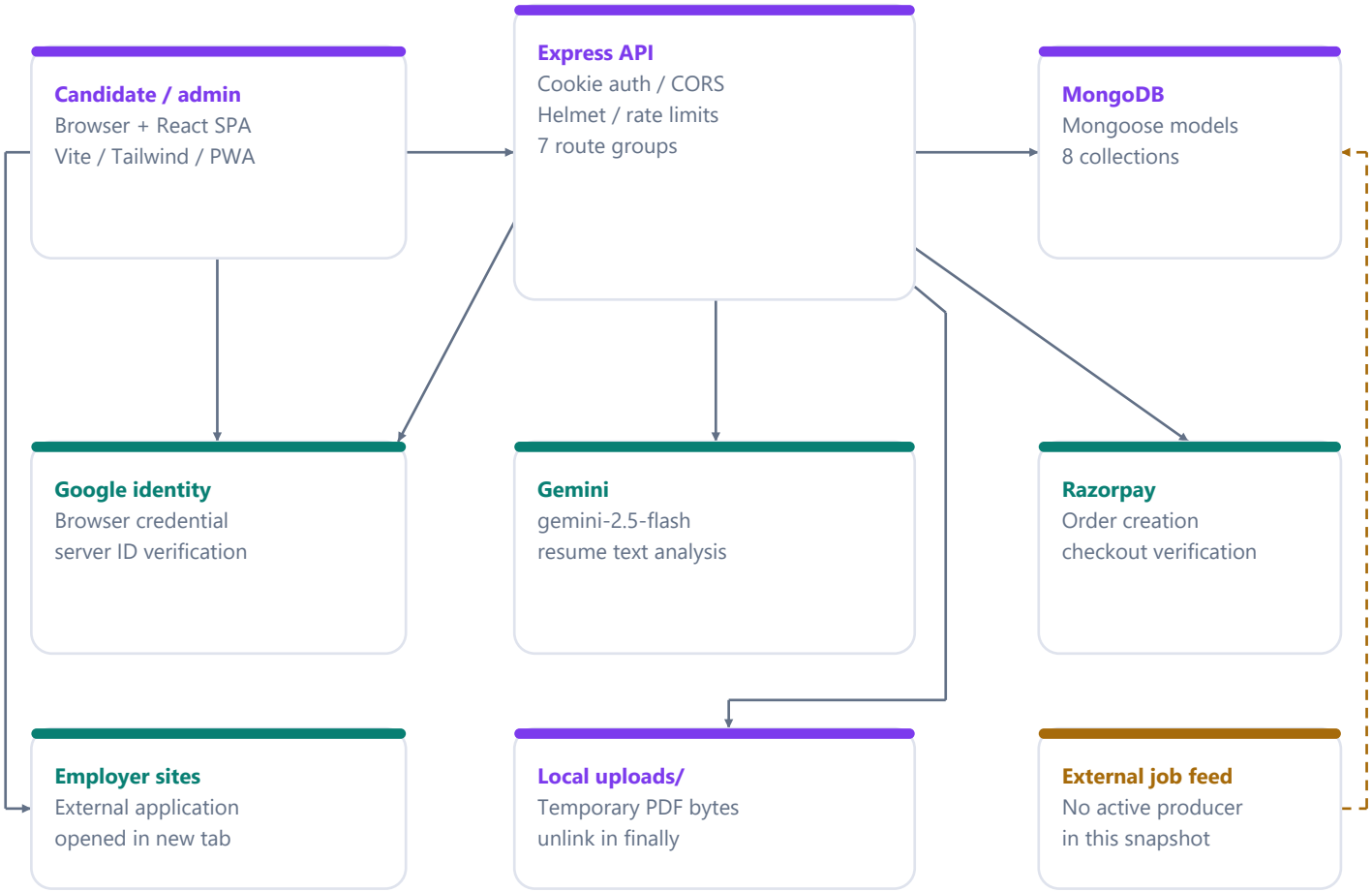
Available management boundaries

No mounted admin APIs were found for user-role editing, payment reconciliation, creating jobs, manually editing jobs, viewing raw feedback submissions or marking feedback Reviewed. The Feedback status field exists, but only aggregated statistics are exposed.

04 / ARCHITECTURE

One web app, one API, shared services

Logical component architecture derived from imports and mounted routes.



Trust and ownership boundaries

The frontend obtains identity credentials and renders product screens; it does not hold the JWT in browser storage. The backend verifies identity, owns entitlement decisions and coordinates persistence. MongoDB stores users, resume content, jobs, applications, feedback, payments and study resources. Gemini receives extracted resume text. Razorpay receives order and checkout data.

Verified	Not established
Express process; MongoDB connection; Google, Gemini and Razorpay integrations	Actual hosting provider, deployment region, network segmentation, database backup policy, TLS configuration and live provider settings.
Temporary local disk and compiled SPA assets	Object storage, CDN provider, worker queue, cache service or separate ingestion service.

Figure 4. Logical architecture only. Connector arrows express calls or intended data inputs, not measured traffic or verified infrastructure placement. Dashed amber indicates an unavailable implementation.

04 / FRONTEND

Routes, state and presentation

A React single-page application built with Vite and BrowserRouter.

Route	Access / purpose
/	Public landing page and Google sign-in.
/aboutMatchora /privacy /pricing	Public product story, sparse active privacy view and checkout entry.
/dashboard	Authenticated workspace; resume status, matches entry, feedback and configured support.
/resume	Authenticated upload, cooldown state and analysis detail.
/recommendations	Authenticated job recommendations, location/platform filters and applied confirmation.
/profile /applications	Authenticated profile/plan display and application list.
/admin /admin/jobs	Authenticated plus server-verified admin; analytics and job deletion.
/pro /terms / unknown paths	No route registered. Pro.jsx is unfinished; About links to /terms. No explicit not-found route.

Provider tree

```
React.StrictMode
  GoogleOAuthProvider
    BrowserRouter
      AuthProvider
        ResumeProvider
          App / routes / Layout
```

AuthProvider calls /api/auth/me and exposes user, loading, logout and refetch. ResumeProvider loads a resume when user.resume is present and exposes resume and Pro state. Dashboard and Profile also perform their own fetches, so state can be refreshed independently rather than through one centralized cache.

Components and browser behavior

Layout supplies a desktop sidebar and mobile drawer with Escape handling and scroll locking. JobCard owns analysis modal state. Recommendations uses AbortController, application-key de-duplication and append pagination. AdminDashboard is lazy loaded; JobManagement is eagerly imported. No state-management package or API client abstraction is required by the current design.

> **PWA and build details**
> vite-plugin-pwa registers an auto-updating service worker and manifest. The active CSS pipeline uses Tailwind 3 through PostCSS, while @tailwindcss/vite 4 is also installed but not configured as a plugin. Offline API data and authenticated offline use are not implemented in reviewed code.

EVIDENCE R30-R40, R43-R44, R49, R51, R56, R59; components/Layout.jsx

05 / DATA FLOW

How a resume becomes structured data

Synchronous request processing with a third-party AI dependency.



Request and storage semantics

Before parsing, the controller checks the stored resume and entitlement. Free users with an existing resume are refused. A future nextUploadAt blocks reupload with 429. Parsed text shorter than 30 characters yields 422. Analysis errors collapse to a generic 500. The work runs inside the HTTP request; no background queue, timeout policy or retry orchestration is shown.

The Gemini call uses model gemini-2.5-flash, temperature 0.2, application/json output and a response schema. The prompt asks for all fields, no invented facts, exact dates and role-specific missing skills. JSON is parsed, defaults are applied and calculateATSScore runs. There is no independent semantic validation of AI claims.

Persisted versus transient fields

Persisted by upload controller	Returned by service but omitted from update
rawText, fileName, canonical skills, education, experience, projects, strengths, missingSkills, preferredRoles, atsScore, suggestions, uploadedAt, nextUploadAt	contact, formatting and rawAnalysis are returned by analyzeResume but not assigned in findOneAndUpdate. rawAnalysis exists in the schema with an empty-object default.

Data handling implications

Deleting the temporary PDF does not delete personal information: rawText and structured history remain in MongoDB and the text has been sent to Gemini. App-level encryption, retention enforcement, user export and deletion workflows are not established.

05 / MATCHING

A transparent matching pipeline

Skill overlap ranks results; experience eligibility filters them.




```
skillScore = round(100 * matchedRequiredSkills / requiredSkills)
no required skills => 0
eligible = userExperienceMonths >= requiredExperienceMonths
keep = skillScore >= MIN_SKILL_SCORE && eligible
```

MIN_SKILL_SCORE is Number(environment value) || 50, so zero also falls back to 50. Scores use exact canonical strings; the helper does not normalize inside calculateSkillScore. Resume skills are canonicalized at upload, while stored job skills must already use compatible names. Duplicated job skills count repeatedly in the denominator and numerator.

Mechanic	Behavior / limit
Experience dates	Expected abbreviated month plus year, or Present. Overlapping roles are summed, not merged. Full month names can produce NaN and filter out every job.
Location	Trimmed to 100 characters and escaped before a case-insensitive MongoDB regex; applied before candidate limits.
Free	LinkedIn, page 1, limit 5. totalJobs still counts the filtered candidate pool, so pagination flags can imply inaccessible later pages.
Pro	Client can choose platform or all; API default is all. UI initially uses linkedin. At most 500 candidates are scored; 25 results per page.
Preferred roles / preferences	The screen needs preferredRoles to start fetching, but role is not sent as an API filter. User preferredRoles, locations, workMode and minimumCTC are not used by recommendJobs.

Interpretation

Example: skills React + Node.js against React + Node.js + Docker yield 67%. A candidate below required experience is excluded even with 100% skill overlap. This is a deterministic matching heuristic, not a calibrated employment suitability model.

05 / SCORING

What the ATS score measures

A custom 0-100 resume completeness and presentation heuristic.

Component	Maximum	Allocation in the helper
Skills	25	1 / 3 / 5 / 8 / 10 / 12 / 15 skills earn 4 / 8 / 12 / 16 / 20 / 22 / 25.
Experience entries	20	1 / 2 / 3+ entries earn 14 / 18 / 20.
Projects	20	1 / 2 / 3 / 4+ entries earn 10 / 15 / 18 / 20.
Education	10	Any education entry earns 10.
Contact presence	10	2 each for email, phone, LinkedIn, GitHub and portfolio.
Formatting	10	AI formatting score capped at 10; no lower bound before the final clamp.
Strengths	10	2 per strength, capped at 10.
Missing skills	-10	Subtract min(number of missing skills, 10).
Absent content penalties	-28	No projects -10; no experience -8; no education -5; fewer than five skills -5.

Important interpretation

Positive component caps add up to 105. The final value is clamped into 0-100 and rounded. More listed entries can raise a score without establishing the quality of the underlying experience. Missing skills and strengths are inferred by the AI, so identical-looking resumes may not yield perfectly stable inputs.

The model sees extracted text rather than the original page image. Assertions about spacing, color, graphics or table layout may therefore be incomplete. Treat formatting advice as a review aid rather than a visual inspection of the PDF.

Illustrative worked calculation

```
8 skills                16
1 experience entry       14
2 projects               15
Education present        10
3 contact flags           6
Formatting score          7
3 strengths               6
4 missing skills         -4
Total / final score      70
```

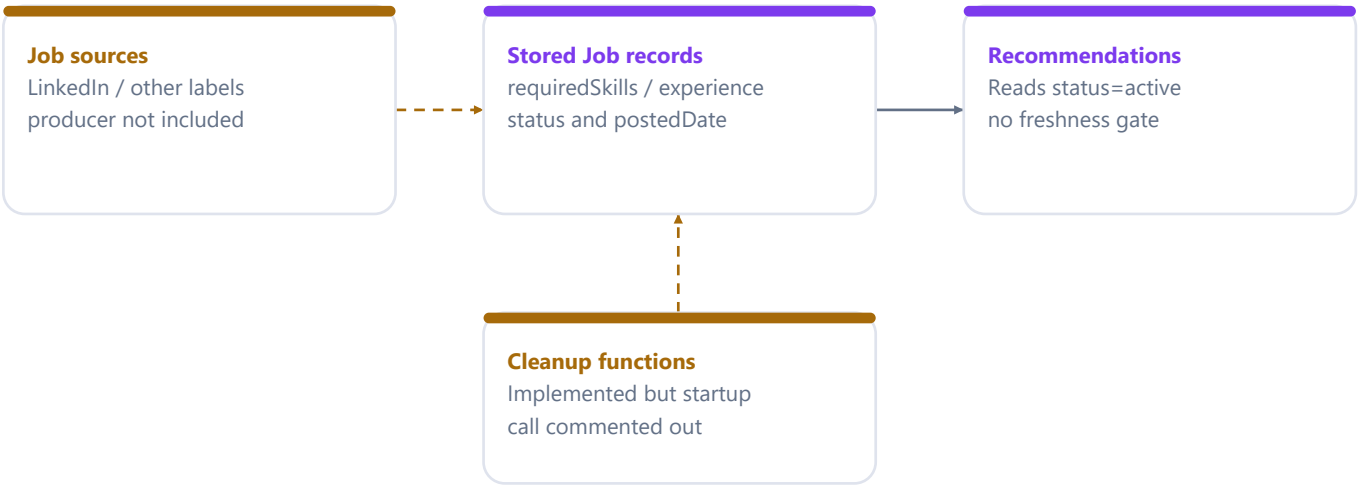
Verification

Isolated checks confirmed the empty input produces 0 and a maximal input is clamped to 100. These checks verify helper arithmetic, not resume quality, model accuracy or real ATS performance.

06 / JOB DATA

Aggregation and lifecycle boundaries

The current repository reads job records; it does not provide an active feed.



What is and is not implemented

searchJobs delegates to scrapeNaukriJobs, which returns an empty array. No scheduled ingestion call, feed credentials, source fetcher, queue consumer or job-create API is present. Platform enums and UI chips are supported labels, not proof of a working integration or current production inventory. A separate external job producer may exist, but is not documented by this repository.

Lifecycle function	Code behavior if explicitly enabled
Expire active jobs	At cron 30 2 * * *, mark active jobs with postedDate older than 20 days expired and set expiredAt.
Delete expired jobs	At 15 2 * * *, delete jobs whose expiredAt is older than 50 days.
Delete old applications	Same 02:15 task; deletes documents with any statusHistory.updatedAt older than 90 days. This is not a last-activity calculation.
Timezone / replicas	No timezone option, leader election, distributed lock or run ledger is configured. Schedule follows process environment; each enabled replica would schedule it.
Freshness helper	isFreshJob checks 36 hours but is not used by recommendJobs. Active status is the actual gate.

Disabled behavior

Both startCleanupCron() and seedSkillResources() are commented out in server startup. Existing comments mentioning 4, 10 or 30 days conflict with constants; the actual values above come from expressions. Do not assume automatic expiry or cleanup is currently occurring.

Recommended feed contract: canonicalize skills, validate dates and platform, assign stable jobKey, upsert against deduplication keys, record provenance and feed time, and monitor lag. No such producer was added during this documentation task.

EVIDENCE R01, R07, R17, R24, R52-R54; utils/isFreshJob.js

07 / DATA DESIGN

Entity relationship map

Eight Mongoose models; ObjectId references are application-level relationships.

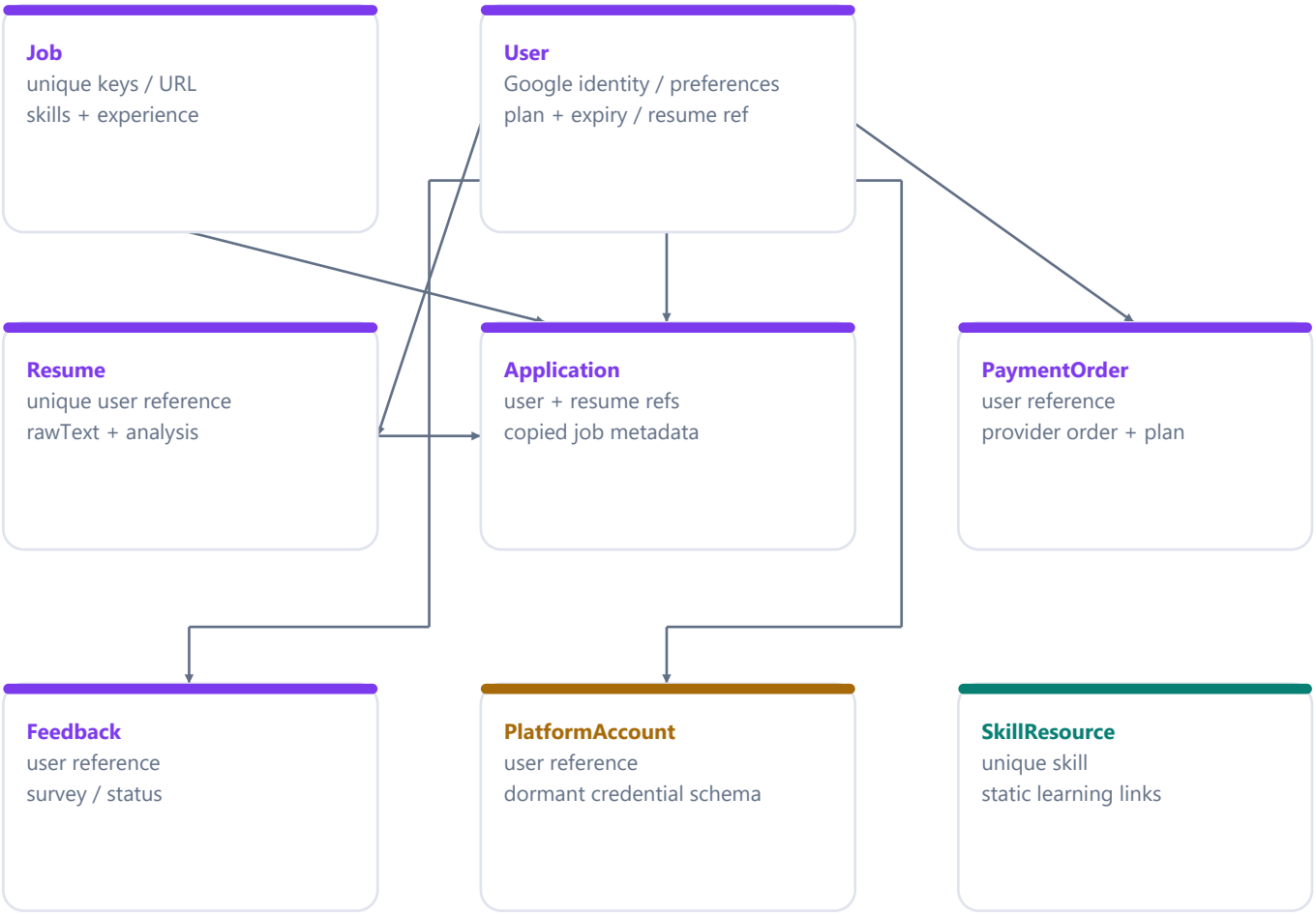


Figure 5. Relationships derive from schemas and controller writes. User → Resume is effectively one-to-one because Resume.user is unique. User has many applications, feedback and payment orders. Application.resume references Resume. The Job-to-Application arrow represents copied job fields: jobId is a string, not an ObjectId foreign key.

Integrity rule	Practical meaning
Schema references are not database foreign keys	No automatic cascades. Deleting a Job does not remove applications, and deleting users would require a designed cleanup workflow.
All models use timestamps	createdAt and updatedAt accompany the model-specific date fields.
Unique indexes declared in source	Actual production index creation, duplicate cleanup and index health were not queried.
SkillResource is independent	Resources are looked up by exact missing-skill string; no relationship index to Resume.

07 / DATA DICTIONARY

Identity and resume records

Field names are exact; grouped fields preserve nesting.

User

Field / constraint	Type and meaning
googleId	String; unique sparse index. Google subject identifier.
name / email	Required trimmed strings; email is lowercase and unique.
picture / lastLoginAt	String default empty; Date default null. lastLoginAt updated at sign-in.
resume	ObjectId ref Resume; default null.
plan / planExpiresAt	free | pro (default free); Date default null.
razorpayCustomerId	String default null; no assignment workflow found.
preferences	preferredRoles[] and preferredLocations[] strings; workMode Remote | Hybrid | Onsite (default Onsite); minimumCTC number default 0. No mounted edit endpoint.

Resume

Field / constraint	Type and meaning
user / fileName / rawText	Required unique User ref; required filename; required extracted personal text.
skills / strengths / missingSkills / preferredRoles / suggestions	Arrays of strings. skills canonicalized and deduplicated by the upload controller.
education[]	degree, institution, location, startDate, endDate, gpaOrPercentage: strings.
experience[]	title, company, location, startDate, endDate: strings; description: string[].
projects[]	title, startDate, endDate, link: strings; technologies and description: string[].
atsScore / rawAnalysis	Number 0-100, default 0; arbitrary Object default {}. rawAnalysis not populated by current upload update.
uploadedAt / nextUploadAt	Date default now; Date for Pro cooldown. Free upload writes null to nextUploadAt.

Persistence and consistency

Resume upsert and User.resume linking are two separate database writes. A failure after the first can leave a stored resume without the expected user link. The update does not request runValidators. Exact deployed validation and migration history are not documented.

AI contact flags and formatting details are not top-level Resume fields. Although the service returns them and rawAnalysis, the controller saves only the listed selected fields.

07 / DATA DICTIONARY

Jobs and application records

Application records preserve a user-reported snapshot of an external opportunity.

Job

Fields	Constraints / semantics
jobKey, title, company, location, jobUrl, postedDate	Required. jobKey globally unique; jobUrl globally unique. title/company/location/jobUrl trimmed. postedDate is Date.
platform	linkedin | naukri | indeed | internshala | foundit | adzuna | others; default linkedin.
description / requiredSkills[]	Description string default empty; trimmed skill strings.
requiredExperienceMonths	Number default 0; no explicit minimum validation.
scrapedAt / status / expiredAt	Date default now; active | expired, default active; Date default null.
Additional indexes	(platform,jobUrl) unique; (platform,jobKey) unique; postedDate descending; requiredSkills; status. Global unique indexes are stricter than compound uniqueness.

Application

Fields	Constraints / semantics
user / resume	Required ObjectId references to User and Resume. Resume ownership is not checked by createApplication.
jobId / company / jobTitle / platform / jobUrl	Required strings. jobId stores the jobKey sent by the frontend. Metadata is client-supplied rather than reloaded from Job.
platform enum	linkedin | naukri | indeed | internshala | foundit. adzuna and others are missing even though Job and the Pro UI support them.
location / aiReason / notes	Strings, default empty.
fitScore / flags	Number 0-100 default 0; recruiterContacted and appliedAutomatically boolean defaults false. No automation path found.
status / statusHistory[]	Saved | Applied | Interview | Offer | Rejected; default Saved. History has status, updatedAt (default now), note. Controller forces new records to Applied.
Index	Unique (user, platform, jobId). Pre-check exists, but concurrent duplicates can still reach a database conflict.

Cross-model compatibility

An Adzuna or others job can be recommended to Pro users but cannot pass the Application platform enum. Application creation returns a generic 500 for this schema validation failure.

07 / DATA DICTIONARY

Payments, feedback and resources

Supporting records and a dormant platform-account schema.

Model	Fields / indexes / current use
PaymentOrder	Required indexed User ref, unique indexed orderId, indexed nullable paymentId, nullable signature; required plan string and amount number; currency default INR; status created | paid | failed (indexed). paidAt and expiresAt default null. Amount is the provider order amount in paise.
Feedback	Required User ref; rating 1-5; issues nonempty enum array; solvesRealProblem Yes | Partially | No; useAgain Definitely | Probably | Maybe | No; npsScore 0-10; mostUsedPlatform enum; wouldPayPremium Yes | Maybe | No; additionalFeedback required, max 2000.
Feedback optional text	likedMost, improvements, missingFeature, weeklyUseReason, recommendationImprovement: trimmed strings, default empty, max 1000. status New | Reviewed, default New. No review mutation endpoint.
SkillResource	Unique required trimmed skill, description default empty; resources[] includes type YouTube | Free | Practice | Docs, required title and URL; isActive default true. Exact skill lookup; no fallback generation enabled.
PlatformAccount	Required User ref, platform enum, email and encryptedPassword; cookies Object default {}; connected default true; lastLogin and lastSynced Dates. Unique (user,platform). No mounted connector, crypto implementation or consuming controller found.

Feedback enum details

Issues: No issues, Resume upload, Resume analysis, Job recommendations, Dashboard/UI, Performance, Other. **Most-used platform:** LinkedIn, Naukri, Foundit, Indeed, Internshala, Unstop, Other. These are survey choices and do not establish supported job integrations.

Dormant schema is not an integration

PlatformAccount suggests an earlier idea for account connections. A field named encryptedPassword does not establish that encryption is implemented. No password collection flow was found in the active app. Do not enable credential storage on the strength of the schema alone.

> **Retention and provider data**
> No retention enforcement for PaymentOrder, Feedback, Resume, User or PlatformAccount is shown. Payment verification stores the provider signature. Production encryption at rest, retention rules, access controls and export policies were not available for verification.

08 / API REFERENCE

API conventions and operation index

All mounted operations, derived from server prefixes and router definitions.

Base URL is the deployed Express origin, supplied to the frontend as VITE_API_URL. The production API hostname is not documented in tracked configuration. API paths below include /api. Protected calls use Cookie: token=<JWT>; browser fetches use credentials: include. There is no bearer-token flow or versioned API prefix in the reviewed code.

Group	Mounted operations
Authentication (3)	POST /api/auth/google GET /api/auth/me POST /api/auth/logout
Resume (2)	POST /api/user/upload-resume GET /api/user/resume
Jobs / applications (3)	GET /api/jobs/recommendations GET /api/jobs/applications POST /api/jobs/applications
Feedback / study (2)	POST /api/feedback GET /api/study/resources
Admin (7)	GET /api/admin/verify GET /api/admin/kpis GET /api/admin/growth GET /api/admin/jobs-analytics GET /api/admin/feedback-stats GET /api/admin/jobs DELETE /api/admin/jobs/:id
Billing (3)	POST /api/payment/create-order POST /api/payment/verify GET /api/payment/status

Response and middleware conventions

Most handlers return JSON with success and optional message, but response envelopes are not fully consistent. The Free reupload 403 omits success; GET /payment/status can return a nullable isPro value because of JavaScript short-circuiting. General API limiting is 300 per 15 minutes; sign-in adds 20 per 15 minutes; upload adds 10 per hour. Limiters use their default key behavior and memory store.

Protected endpoints return 401 for a missing/invalid/expired cookie or missing user. Admin adds 403; study adds 403 for non-Pro access. Controller exceptions generally return 500. JSON parse and Multer errors have no custom global formatter, so middleware failures need not follow the JSON envelope.

> **Unmounted operations**
> No /api/applications prefix is mounted despite controller comments. The application status PATCH and deletion routes are commented out. GET / returns plain text "this is home page" and is not a database readiness endpoint.

08 / API / AUTHENTICATION

Sign-in and session endpoints

Cookie-based authentication with Google ID-token verification.

POST /api/auth/google | Public + auth limiter

Content-Type: application/json. **Required body:** token, the Google ID token returned by the frontend identity control. The server verifies it against GOOGLE_CLIENT_ID, looks up the user by email, creates or updates identity fields and lastLoginAt, then issues a seven-day JWT.

```
Request: {"token": "<google-id-token>"}
200: {"success": true, "user": {"_id": "...",
      "name": "Example User", "email": "user@example.com",
      "plan": "free", "resume": null, "...": "..."}}
Set-Cookie: token=<jwt>; HttpOnly; Max-Age=604800
```

400: missing token. **429:** limiter. **500:** failed verification or other caught exception; message is err.message. Invalid credentials are not separately mapped to a 401 in this handler.

GET /api/auth/me | Authenticated

No request body or query. Returns **200 {success:true,user}**, where user is the full loaded Mongoose User document. Includes identity, resume reference, preferences, plan and timestamps. The response is not a deliberately minimized public profile projection.

POST /api/auth/logout | Authenticated

No request body. Clears the token cookie with matching flags and returns **200 {success:true,message:"Logged out successfully"}**. Missing or expired auth is rejected before the logout handler. Clearing a browser cookie does not revoke other copies of the signed token.

Setting	Implementation
JWT payload / expiry	id and email; expiresIn 7d. Signed with JWT_SECRET.
Cookie in production	HttpOnly=true, Secure=true, SameSite=None, seven-day maxAge.
Cookie otherwise	HttpOnly=true, Secure=false, SameSite=Lax.
Middleware	Reads req.cookies.token; jwt.verify; User.findById(decoded.id); attaches req.user.
CORS	CLIENT_URL origin and credentials=true. No separate origin verification or CSRF token middleware found.

Integration note

Use the same Google client ID on frontend and backend, register the actual frontend origins with Google, and verify cookie behavior in the deployed domain arrangement. Third-party cookie restrictions can affect cross-site sessions. Provider and deployment configuration were not checked.

08 / API / RESUME

Upload and retrieve resume analysis

Both endpoints require a valid session cookie.

POST /api/user/upload-resume

Content-Type: multipart/form-data. One file field named **resume**. Browser forms should let the browser set the multipart boundary. Order of middleware: upload limiter, auth, upload.single("resume"), controller.

```
curl -X POST "$API/api/user/upload-resume" \
  -b cookies.txt -F "resume=@example-resume.pdf"

200: {"success": true, "message": "Resume analyzed successfully.",
      "resume": {"_id":"...", "skills":["React"],
        "atsScore":70, "nextUploadAt":"<ISO-date-or-null>",
        "...":"other Resume fields"}, "isPro":true}
```

Status	Trigger / response detail
400	No req.file: success=false, message Please upload a resume.
403	Existing resume and non-Pro: message explains one Free upload; no success key.
422	Extracted text missing or fewer than 30 trimmed characters; asks for text-based PDF.
429	Future nextUploadAt: success=false, message plus nextUploadAt. Limiter-generated 429 has a different message and no cooldown timestamp.
500	Parsing, Gemini, persistence or other controller error: success=false, Resume upload failed. Middleware errors are not normalized.

GET /api/user/resume

No input parameters. Finds the Resume by the authenticated user ID. **200:** {success:true,message:"Resume fetched successfully.",resume,isPro}. **404:** Resume not found. **500:** success=false with the exception message. The result includes rawText, so access and response handling should protect personal information.

Side effects and limits

A successful upload overwrites the user-linked Resume through upsert, updates User.resume and sets the Pro cooldown. Temporary-file deletion is attempted after the response path. No OCR, antivirus, job queue, upload byte limit, server type filter or durable original-file storage is configured.

> **Entitlement discrepancy**
> isProActive treats plan=pro with no expiry as active here. Recommendations, study, payment status and sidebar logic require a future expiry. A malformed or legacy Pro record can therefore have inconsistent access.

08 / API / RECOMMENDATIONS

Get resume-matched jobs

GET /api/jobs/recommendations - authenticated.

Query	Accepted behavior
page	Pro: parseInt, minimum 1, default 1. Free: forced to 1.
platform	Pro: supplied value or all; no enum validation. Free: forced linkedin.
location	String trimmed and truncated to 100 characters. Empty means no location filter. Escaped case-insensitive substring regex.
limit	Client parameter ignored. Fixed 25 for Pro and 5 for Free.
role / preferred roles	No role query is consumed. Resume skills and experience drive matching.

```
GET /api/jobs/recommendations?page=2&platform=all&location=Pune

200: {"success":true,"jobs":[{
  "_id":"...","jobKey":"source-123","title":"Backend Developer",
  "company":"Example Co","platform":"linkedin",
  "requiredSkills":["Node.js","Docker"],
  "skillScore":50,"matchedSkills":["Node.js"],
  "missingSkills":["Docker"],"eligibility":{"experience":{
    "requiredMonths":12,"requiredYears":1,"userMonths":18,
    "userExperience":"1 Years 6 Months","eligible":true}},
  "...":"other stored Job fields"}],
  "page":2,"limit":25,"totalJobs":40,"totalPages":2,
  "hasNextPage":false,"hasPreviousPage":true,
  "isPro":true,"plan":"pro"}
```

Errors and count semantics

404: no resume for the authenticated user. **500:** Failed to fetch recommendations. Common auth and limiter errors also apply. totalJobs is the number surviving score and experience gates within the latest bounded 100/500 candidates, not the size of the complete collection. totalPages is at least 1 even for zero results.

The backend does not exclude already applied jobs. The browser hides application keys locally after reading /api/jobs/applications. Counts and visible cards can therefore disagree. Candidate pooling before scoring can omit an older strong match when the newest matching-skill pool fills the cap.

> **Worked score**
> The example is synthetic: one matched requirement out of two yields 50%; 18 months meets a 12-month requirement. Under the default threshold of 50 the job is eligible. API output is not a prediction of selection or hiring.

08 / API / APPLICATIONS

Record and list applications

Actual mount prefix is /api/jobs/applications.

GET /api/jobs/applications | Authenticated

No query or body. Returns **200 {success:true,applications:[Application]}**, sorted newest createdAt first and scoped to req.user.id. No pagination or field projection is applied. **500:** Failed to fetch applications.

POST /api/jobs/applications | Authenticated

Input field	Required / handling
jobId, company, jobTitle, platform, jobUrl	Controller requires nonempty values; jobId is the external jobKey, not Job._id.
resume	Schema requires an ObjectId reference; controller does not include it in its explicit missing-field check and does not verify ownership.
location, fitScore, appliedAutomatically, aiReason	Accepted from the client. Schema defaults and constraints apply.
status	Read from body but ignored: controller forces Applied.

```
Request: {"resume":"<resume-object-id>","jobId":"source-123",
 "company":"Example Co","jobTitle":"Backend Developer",
 "platform":"linkedin","jobUrl":"https://example.com/job/123",
 "location":"Pune","fitScore":67,"appliedAutomatically":false}

201: {"success":true,"application":{
 "status":"Applied","statusHistory":[{
  "status":"Applied","note":"Application created",
  "updatedAt":"<ISO-date>"}],"...":"other Application fields"}}
```

Idempotency and failure behavior

Before insertion, the controller checks (user, platform, jobId). An existing record returns **200** with success, application and message Application already exists. A new record returns **201**. Missing explicitly checked fields return **400**. Schema validation and concurrent uniqueness failures fall through to **500** Failed to create application.

> **Unsupported mutations**
> deleteApplication exists as an owner-scoped controller, but its route is commented out. There is no mounted application status update. Do not integrate clients against commented routes or the misleading /api/applications controller comments.

Recommended hardening: load the authenticated user resume and trusted Job server-side; validate allowed platforms and URLs; remove client authority over automation flags and fitScore; handle duplicate-key errors as a safe repeat operation.

08 / API / FEEDBACK AND STUDY

Feedback submission and learning links

Survey persistence is active; study support has an API-only implementation.

POST /api/feedback | Authenticated

Required JSON fields: rating, issues, solvesRealProblem, useAgain, npsScore, mostUsedPlatform, wouldPayPremium, additionalFeedback. Optional text fields: likedMost, improvements, missingFeature, weeklyUseReason, recommendationImprovement. User ID is taken from the session.

```
{"rating":4,"issues":["No issues"],"solvesRealProblem":"Yes",
 "useAgain":"Probably","npsScore":8,"mostUsedPlatform":"LinkedIn",
 "wouldPayPremium":"Maybe","additionalFeedback":"Useful matches."}
```

201: {success:true,feedback:Feedback}. **400:** missing field list or schema validation messages joined together. **500:** Failed to submit feedback. rating accepts 1-5 and npsScore accepts 0-10; code does not explicitly require integer values. Text limits and enums are in the schema (page 19).

The frontend makes No issues exclusive with other issues; the backend schema only validates enum members and a nonempty array, so that exclusivity is not enforced by the server. No feedback list, update or review-state mutation route is mounted.

GET /api/study/resources | Authenticated + active Pro

Finds the user Resume and iterates resume.missingSkills. For each string it queries SkillResource with the same skill and isActive=true. Found results include skill, description, resources and source:"static". Unknown skills are skipped. The commented Gemini fallback is not active.

```
200: {"success":true,"missingSkills":["Docker"],
 "resources":[{"skill":"Docker","description":"...",
  "resources":[{"type":"Docs","title":"...","url":"https://..."}],
  "source":"static"}]}
```

Status / case	Response behavior
403 / inactive Pro	success=false, Upgrade to Pro to access Study Support, upgradeUrl=/pricing.
404 / no resume	success=false, message asks for a resume first.
200 / no missing skills	success=true, explanatory message, resources=[]; missingSkills is omitted on this branch.
500	success=false, Failed to fetch study resources.

Availability boundary

The /pro navigation item is commented out; no route is registered. Pro.jsx has undefined identifiers and no usable rendered return. Static seeding is also disabled on startup, so API data depends on existing database contents.

EVIDENCE R15-R16, R27-R28, R30, R52, R55-R56, R58, R65-R66

08 / API / ADMIN

Admin analytics and job management

All seven endpoints require auth followed by the ADMIN_EMAILS allowlist.

Method + path below /api/admin	Success response fields / behavior
GET /verify	200 {success:true,isAdmin:true}. Reaching handler means both guards passed.
GET /kpis	200 {success:true,kpis:{totalUsers,activeUsers30d,totalJobs,totalApplications,totalResumes}}. totalJobs is active jobs.
GET /growth?days=30	200 {success:true,growth:{userGrowth,applicationsOverTime,jobsOverTime}}. Arrays of {date:"YYYY-MM-DD",count}; dense UTC-day series. Number defaults to 30 and clamps between 7 and 90; integer validation absent.
GET /jobs-analytics	200 {success:true,jobsAnalytics:{byPlatform:[{platform,count}],topCompanies:[{company,count}],jobsToday}}. Platform/company counts include all statuses.
GET /feedback-stats	200 {success:true,feedbackStats:{totalFeedback,pendingFeedback,ratingDistribution:[{rating,count}],avgRating,avgNps}}. Averages null if aggregation has no result.
GET /jobs	200 {success:true,jobs:[...]} active jobs sorted postedDate descending. Selected fields: _id,title,company,location,platform,jobUrl,postedDate,requiredSkills,requiredExperienceMonths,status.
DELETE /jobs/:id	200 {success:true,message:"Job deleted successfully.",jobId}. Permanently deletes the matching Job by MongoDB ID.

Errors

Common **401** for missing/invalid auth; **403** Admin access required for an email outside the allowlist. Each async handler maps caught errors to **500** with a route-specific load/delete message. Delete returns **404** for a well-formed ID that does not exist. A malformed MongoDB ID is caught as **500**, rather than validated to 400.

Metric and access caveats

There is no pagination or result cap for GET /jobs. Deletion does not cascade to application snapshots. There is no admin audit record, soft-delete, user management or billing operations endpoint in the reviewed routes. ADMIN_EMAILS is trimmed, lowercased and read on each check; an empty list denies all admins.

> **NPS semantics**
> avgNps is simply the mean of submitted npsScore values. Conventional Net Promoter Score requires a promoter-minus-detractor calculation; the current API does not implement that calculation.

08 / API / BILLING

Create, verify and inspect entitlement

Razorpay Standard Checkout with a local PaymentOrder record.

POST /api/payment/create-order | Authenticated

Required JSON: {plan:"monthly"} or {plan:"yearly"}. PLANS selects the server amount and duration. Razorpay receives amount × 100 in paise and INR currency. A local PaymentOrder is created with status created and the provider order ID.

```
200: {"success":true,"orderId":"order_example","amount":3000,
 "currency":"INR","keyId":"<public-razorpay-key-id>",
 "plan":"monthly","paymentOrderId":"<object-id>",
 "user":{"name":"Example User","email":"user@example.com"}}
```

400: Invalid plan selected. **500:** Failed to create payment order. Amount above is illustrative only; runtime PRO_monthly_rate and PRO_yearly_rate control actual charges. No configured values were read.

POST /api/payment/verify | Authenticated

Required JSON: razorpay_order_id, razorpay_payment_id, razorpay_signature. Looks up the local order by orderId and authenticated user, rejects already-paid records, and derives the plan from that record. A client plan field is ignored.

```
expected = HMAC_SHA256(secret, order_id + "|" + payment_id)
if signature matches:
  expiryBase = future active Pro expiry, otherwise now
  expiresAt = expiryBase + (30 or 365 days)
  save PaymentOrder paid, then update User Pro

200: {"success":true,"message":"Pro plan activated successfully!",
      "expiresAt":"<ISO-date>"}
```

400: missing details, already processed, invalid stored plan or signature mismatch. Mismatch marks order failed. **404:** no owned local order. **500:** Payment verification failed. Comparison uses normal string inequality, not a timing-safe comparator.

GET /api/payment/status | Authenticated

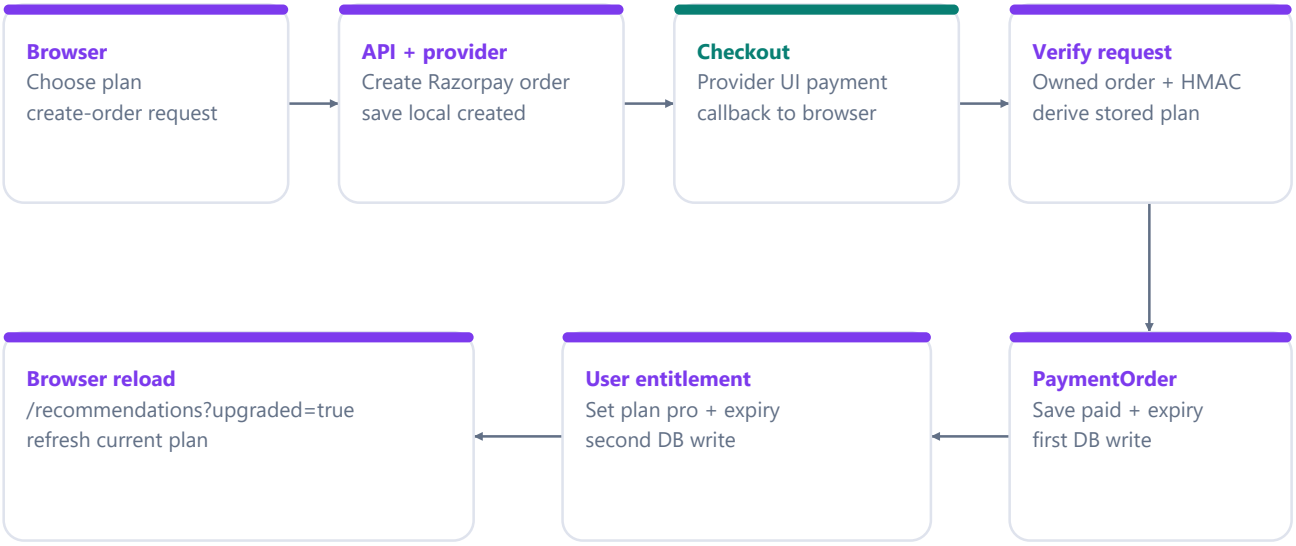
200: {success:true,plan,isPro,planExpiresAt}. isPro requires plan pro and a future expiry; it can be null when plan is pro but expiry is null. **500:** Failed to get payment status. The endpoint does not automatically rewrite an expired user.plan back to free.

> **Integration boundary**
> No webhook, provider payment-status fetch, explicit capture operation, refund endpoint or recurring subscription schedule was found. Successful live charging and provider capture settings were not tested.

09 / BILLING WORKFLOW

Entitlement activation and recovery

Source-derived sequence and production reliability considerations.



What the code protects

The order is tied to the session user; price and duration come from server-side PLANS; verification uses the stored order plan and checks the provider signature. Serial retries after an order is marked paid are rejected. Existing future Pro expiry is extended rather than discarded.

Where reconciliation is needed

Finding	Consequence / recommended control
Separate persistence writes	If paid order save succeeds and User update fails, retry is rejected as already processed. Use an atomic transaction or explicit recovery path.
Read-check-write processing	Concurrent valid verify requests can race; no atomic status transition or transaction guarantees one activation. Concurrent distinct renewals can lose time.
Browser callback dependency	Closed tab or network failure can leave a paid provider order without entitlement activation. Add authenticated provider events and reconciliation.
Independent price display	Pricing hardcodes INR 30 / 299 while server env controls charge. Verify parity. 12 × 30 - 299 is INR 61; current savings copy says INR 66.

Recommended operational rule: reconcile provider captured payments with local orders and user expiry before granting, reversing or extending entitlements manually. Maintain an audit trail and establish a tested refund policy. These processes are not currently documented in source.

EVIDENCE R11-R12, R26, R40; D03 official Razorpay integration guidance

10 / CONFIGURATION

Environment configuration inventory

Names are exact and case-sensitive; examples contain no real secrets.

Backend variable	Purpose / validation boundary
PORT	Express listener; no code default or startup validation.
MONGO_URL	Mongoose connection URI. Never expose through VITE_ variables.
JWT_SECRET	JWT signing and verification secret. Rotation/revocation process not implemented.
GOOGLE_CLIENT_ID	Audience for server ID-token verification; same client as frontend.
CLIENT_URL	Single allowed CORS origin with credentials.
GEMINI_API_KEY	Google GenAI key; loaded by config/gemini.js.
NODE_ENV	Use production for Secure / SameSite=None session cookies.
ADMIN_EMAILS	Comma-separated lowercase-normalized admin emails. Empty denies all.
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET	Provider order client and HMAC verification. Key ID is returned to checkout; secret remains server-side.
PRO_monthly_rate / PRO_yearly_rate	Backend price amounts in rupees, multiplied by 100 for Razorpay; no value/range validation.
MIN_SKILL_SCORE	Recommendation threshold; numeric value or fallback 50. Zero cannot be selected with current || logic.

Frontend variable	Purpose
VITE_API_URL	API origin at build time. Required for most screens; a few use localhost:5000 fallback.
VITE_GOOGLE_CLIENT_ID	GoogleOAuthProvider client ID. Public identifier.
VITE_SUPPORT_EMAIL / VITE_WHATSAPP_SUPPORT_NUMBER	Optional dashboard support links. Values are shipped to the browser.
VITE_PRO_MONTHLY_RATE / VITE_PRO_YEARLY_RATE	Referenced in Recommendations with 99/999 fallback, but not the authoritative Pricing display or backend charge. Do not treat these as a single pricing source.

Configuration hygiene

GOOGLE_CLIENT_SECRET appears in README examples but is not referenced in runtime code. No .env.example, typed config contract or secret values were tracked in the inspected tree. Do not infer deployed configuration from documentation defaults.

Load environment values before module initialization. config/gemini.js calls dotenv.config(), but several imported modules capture environment values at top level. Prefer explicitly preloading the environment in the process command and validating required values before listening.

EVIDENCE R01-R03, R07, R12, R19-R21, R31, R35, R38, R40, R61-R62

11 / DEVELOPMENT

Run and build a local copy

Recommended setup procedure; no app dependencies were installed in this review.

Prerequisites

Use a maintained Node.js release satisfying all lockfile engine constraints. The pinned Vite 8.1.3 lockfile requires ^20.19.0 or >=22.12.0; Mongoose 9.7.3 requires >=20.19.0. A compatible Node 22+ environment is a practical baseline, subject to local validation. Supply MongoDB, Google OAuth, Gemini and Razorpay test credentials.

```
git clone https://github.com/abhinav84094/Matchora.git
cd Matchora
git checkout d72d00380934bf62b9e0e31eeaa24023ec461032

# Terminal 1 - from backend/
npm ci
# create backend/.env with backend variables
npm start

# Terminal 2 - from frontend/
npm ci
# create frontend/.env.local with VITE_ variables
npm run dev
```

Frontend dev and preview bind to 127.0.0.1. Match CLIENT_URL and the Google authorized JavaScript origin to the actual development frontend URL. Set VITE_API_URL to the backend origin without an /api suffix. Keep all provider secrets in the backend environment.

Build and verification commands

```
# frontend/
npm run lint
npm run build
npm run preview

# backend/ - syntax is not an integration test
node --check server.js
# npm test currently exits 1: "no test specified"
```

Package / script	Observed behavior
Backend start	nodemon server.js, even for npm start. No separate production or dev script.
Frontend build	vite build; resulting static assets go to dist by Vite convention.
Testing	No repository test suite or runner configuration found. Frontend lint exists; backend test is a placeholder.
Job and study data	No jobs are loaded by setup. Study seed function exists but is not invoked. A blank DB will not produce job recommendations or static learning links.

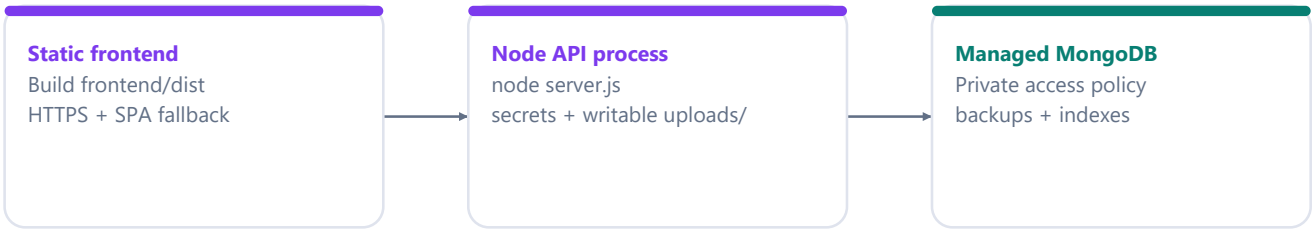
Documentation-only task

The source was retrieved into an isolated inspection directory, not checked out into or written back to the upstream repository. Setup commands above are instructions for maintainers; they were not executed against a running Matchora service.

11 / DEPLOYMENT

Deployment topology and release procedure

Recommended operating pattern; actual host and deployment pipeline are unverified.



Release checklist for maintainers

Step	Concrete verification
1 / Build	Install from the lockfiles, set VITE_ values before build, run lint and build, retain revision and artifact identifiers. Build and lint were not executed during this review.
2 / API environment	Validate all backend variables, provider test/live mode, price values and authorized origins. Launch with a production process manager or managed service; no manifest is included.
3 / Static routing	Preserve assets and crawl files before the SPA fallback. public/_redirects contains /* /index.html 200; host compatibility and actual rule application are not established.
4 / Connectivity	Verify DB readiness, provider calls and disk write permissions. The server listens before DBconnection resolves; connection failures are logged without failing the process.
5 / Cookies and proxy	Check HTTPS, NODE_ENV=production, CORS credentials and browser cookie behavior. trust proxy=1 assumes one trusted proxy hop; verify the actual network path.
6 / Smoke test	Use a test user and provider test mode to verify sign-in, upload, recommendations, application recording and entitlement activation, plus non-admin denial.
7 / Observe and rollback	Watch errors, latency, feed age and payment reconciliation. Keep a known-good frontend/API revision and compatibility-aware DB restore plan.

Missing infrastructure definitions

No Dockerfile, Compose, platform deployment manifest, infrastructure-as-code, CI workflow, .env example or explicit health/readiness endpoint was found in the pinned tree. The redirect file alone does not identify the hosting provider.

PWA releases need cache/update checks so old clients do not call incompatible API contracts. Restrict caching of personalized API responses and verify service-worker update behavior on an installed client. Current offline and cache behavior was not exercised.

EVIDENCE R01, R48-R51, R61; repository tree

12 / OPERATIONS

Maintain service and recover safely

Recommended runbook based on implementation behavior.

Signal / incident	First checks and recovery direction
Sign-in fails or never completes	Check Google client ID/audience, allowed origin, API reachability and auth response. Confirm cookie flags, credentials inclusion and browser policy. Add frontend timeouts for a stuck auth check.
Uploads fail	Distinguish 403 entitlement, 422 unreadable text and 429 cooldown/rate limit. Then inspect uploads/ permissions, available disk, PDF parser and Gemini errors. Do not log raw personal resume text.
No recommendations	Verify Resume skills and dates; check preferredRoles for frontend fetch gating. Confirm compatible Job.requiredSkills, sufficient experience, platform/location filter and active inventory.
Jobs appear stale	Measure newest postedDate/scrapedAt and last producer run. No active ingestion or cleanup is supplied here; identify the actual external producer before restarting anything.
Paid but not Pro	Compare provider payment/order state with local PaymentOrder and User expiry. Avoid blind replay after paid status; current serial retry rejects it. Establish an audited reconciliation action.
Admin denied	Confirm signed-in email and ADMIN_EMAILS normalization. Empty or missing allowlist denies all; ordinary User records do not carry an admin role.
Slow requests / many 429s	Measure parsing/AI latency, database queries, pool size and shared egress IP effects. In-memory limits do not coordinate replicas and reset on process restart.
Disk or retention pressure	Inspect orphan uploads after crashes. Cleanup finally cannot run after process termination. Review retention rules before enabling destructive cron functions.

Monitoring to add

Track request rate, p95 latency and errors by route; MongoDB connection and query health; Gemini latency, failures and cost; upload rejection counts; recommendation pool size and empty-result rate; job-feed lag; payment created-to-paid-to-active mismatches; disk usage and limiter events. No metrics exporter, alert definitions or structured tracing was found.

Backups, privacy and rollback

Define encrypted backups, retention and tested restores for users, resumes, jobs, applications, feedback and orders. Database restoration can roll back entitlements and payment history, so reconcile provider records after restore. Define an account deletion/export workflow and operator authorization for sensitive data. These policies are not specified by source.

> **Cron activation is a change**
> Before enabling cleanup, confirm intended retention, timezone and single scheduler ownership. The application cleanup predicate matches any old history entry and can delete a record with newer activity. Correct and test that policy before activation.

EVIDENCE R01-R04, R07-R11, R17-R21, R61; recommended operator practices

13 / SECURITY

Implemented controls and trust boundaries

Static review of safeguards, with production behavior explicitly unverified.

Area	Implemented control	Residual boundary
Identity	Google ID-token verification against audience; signed JWT; user lookup on each protected call.	No token revocation, session registry or refresh flow. Logout clears only the cookie.
Cookies	HttpOnly, seven-day lifetime; Secure and SameSite=None in production.	Cross-site cookie behavior and CSRF defenses need validation. CORS is not a complete authorization or CSRF mechanism.
Authorization	Owner-scoped resume and application list; admin email allowlist; Pro middleware for study.	Application input accepts client resume reference and job metadata. Entitlement predicates differ by feature.
HTTP protection	Helmet, single-origin credentialed CORS, general/auth/upload rate limiters.	Header policy at deployed frontend not verified. Default in-memory limiter store does not coordinate replicas.
Files	Auth before upload disk storage; temporary deletion attempted in finally; PDF text extraction.	No server byte/type/signature filter, random filename, malware scan or parser resource isolation.
Billing	Owned order lookup, server-selected plan, HMAC check, serial paid-state guard.	No transactional activation, atomic idempotency, webhook reconciliation or capture-state confirmation.
Data	Secrets read from env; .env patterns ignored in source.	No app-level encryption or retention/delete/export workflow established. Database/platform encryption unverified.

Sensitive-data path

Google identity fields enter User; resume text reaches Gemini and is saved as rawText; payment IDs and verification signature enter PaymentOrder; feedback stores free text tied to the user. API responses often return entire documents. Apply least-privilege database access, response minimization and redacted logs around these paths.

> **Live privacy gap**
> The live /privacy view and active component render only navigation and a short agreement sentence. The long policy body in the file is commented out. Users are not shown a substantive explanation of processing, providers, retention or rights in the inspected view.

No exploitation, brute-force attempts, malicious uploads, third-party credential access or production data probing was performed. Findings are code-supported review observations and proposed hardening, not claims of a demonstrated compromise.

13 / REVIEW FINDINGS

Prioritized engineering findings

Suggested priorities, not an externally scored vulnerability assessment.

Priority	Evidence-based finding	Recommended next action
P1	Upload route lacks server file size/type restrictions; PDF parsing and AI are synchronous. R18/R04.	Enforce byte limit, signature/type policy, unique safe names, cleanup sweeper and bounded parser/AI resources.
P1	Paid order and user entitlement persist separately; paid-state check is non-atomic. R11.	Atomic transition + transaction/recovery state; provider event reconciliation; test retries and concurrency.
P1	Privacy page is sparse; rawText is retained and sent to Gemini. R04/R41 + live.	Publish accurate processing/provider/retention terms and implement authorized export/deletion lifecycle.
P1	Production SameSite=None cookies lack explicit CSRF tokens/origin enforcement. R01/R02.	Threat-model each mutation and add server-checked CSRF/origin protection appropriate to deployment.
P2	Full month date strings yield NaN; overlapping experience is double-counted. R07.	Normalize/validate dates and merge time intervals; test parser output variants and invalid dates.
P2	Adzuna/others mismatch between Job and Application; client metadata and resume accepted. R24/R25/R09.	Align platform contract and derive trusted job/resume data on the server.
P2	Pro missing-expiry access differs by endpoint; UI pricing differs from charge configuration. R21/R08/R12/R40.	Centralize entitlement and pricing rules; validate config; remove misleading unlimited/savings copy.
P2	DB connection failure is swallowed and API listens before readiness. R01/R61.	Fail startup on invalid config/DB dependency; add liveness/readiness and controlled shutdown.
P2	Job ingestion absent; cleanup and resource seed disabled. R01/R53.	Document actual producer ownership and feed SLA; add tested scheduling and retention rules.
P3	No tests/CI; incomplete Pro/terms routes; shared SEO metadata and no 404 route. Tree/R30/R45.	Build a release gate, finish/remove dead routes and introduce route-specific public metadata.

P1: address before increasing exposure or relying on unattended billing/data processing. P2: correctness and operational reliability. P3: maintainability and product polish. Priorities depend on actual usage and deployment; no production impact was measured.

EVIDENCE Sources shown per finding; detailed source register on pages 38-39

14 / SEO

Search visibility: verified and unknown

The code supplies a useful baseline; indexing and ranking remain unverified.

SEO item	Evidence / status
Title and description	CODE VERIFIED + LIVE OBSERVED: homepage metadata present. Same global title on Pricing/About; live Pricing canonical points to homepage.
Canonical and social cards	Global homepage canonical, og:url, title/description and Twitter summary card in index.html. No og:image or twitter:image defined there.
Structured data	Organization, WebSite and SoftwareApplication JSON-LD graph present in source and live DOM. Rich-result eligibility or validation was not established.
robots.txt / sitemap.xml	Files exist in source. Sitemap lists home, About, Pricing, Privacy. Browser navigations showed blank application views; raw HTTP status/content-type could not be confirmed. Do not treat file presence as proof of correct live delivery.
Public routing	BrowserRouter SPA; catch-all rewrite file exists. No explicit 404 route, per-route metadata manager, SSR or prerender setup found.
Search Console / indexing	Earlier chat reported verification and sitemap submission. No current account access or indexing evidence was available; prior chat claims were not promoted to verified status.
Performance	No Lighthouse/PageSpeed run, field Core Web Vitals or performance trace captured. No speed score or SEO grade is asserted.

Recommended SEO work in order

Delivery: fetch robots.txt and sitemap.xml outside an active service worker and verify HTTP status, content type and raw body. Ensure static files precede SPA fallback. **Public routes:** assign unique titles, descriptions and self-canonicals where distinct pages should be indexed; return useful 404 behavior for unknown URLs. **Rendering:** validate rendered content with Search Console; consider prerendering the public marketing pages.

Content accuracy: reconcile free/Pro and application-tracking claims, complete privacy/terms, and add a real share image. **Measurement:** collect mobile performance and Search Console indexing/coverage evidence before making ranking claims.

> **Crawl request is not indexing**
> Google states that requesting a crawl does not guarantee inclusion in search results. Current indexing status must be checked in Search Console; a previous priority-queue message is not proof that Matchora is indexed.

Primary guidance: D01 Google recrawl documentation; D02 JavaScript SEO basics. These recommendations are inferred from that guidance and the observed SPA implementation.

15 / TESTING

Verification performed and release gaps

Checks here are bounded; they do not establish full production readiness.

Performed during this review	Result / interpretation
Source inventory	91 text blobs retrieved at pinned SHA; tree not truncated. Binary/vector assets inventoried. All source reads were local or GitHub GET operations.
Backend syntax	43 JavaScript files passed node --check. This parses syntax only; imports, secrets and runtime integrations are not exercised.
Matching arithmetic	2 of 3 skills produces 67; no required skills produces 0. Isolated functions used with the database import stubbed.
Experience characterization	Jan 2024 to Jan 2025 produces 12 months; January 2024 to January 2025 produces NaN. The latter confirms a current defect, not desired acceptance behavior.
ATS and plan characterization	Empty ATS input gives 0; maximal input clamps to 100; Pro helper with missing expiry returns true. Last behavior exposes entitlement inconsistency.
Public browser inspection	Home, About, monthly/yearly Pricing and sparse Privacy observed. Raw crawl file delivery and authenticated journeys remain unverified.
Not executed	npm ci, lint, frontend build, dependency vulnerability scan, integration suite, E2E suite, real OAuth, AI request, payment, DB restore or production load test.

Proposed release acceptance suite

Area	Essential scenarios
Identity and access	Missing/invalid token; valid Google audience; cookie flags; logout; admin allow/deny; cross-origin mutation handling.
Resume	5 MB boundary enforced server-side; wrong type; malformed/scanned PDF; concurrent first upload; cooldown; parser/AI failure and cleanup.
Recommendations	Canonical aliases; empty pool; exact threshold; malformed/full month/overlapping dates; location escaping; Free pagination; Pro source enums.
Billing	Test-mode success/failure, amount parity, owned order, bad signature, duplicate/concurrent verify, user-write failure and provider callback loss.
Operations and UI	Build/lint gate; auth loading timeout; mobile drawer/modals; original job links; 404s; crawl file responses; backups/restore and old PWA client.

The current backend npm test is a deliberate failure placeholder, and no CI workflow or tracked test suite was found. The 50 completed checks are 43 syntax checks plus 7 isolated helper observations; they are not a substitute for the acceptance suite above.

EVIDENCE Local audit-results.json; R50-R51; repository tree; L01-L04

16 / LIMITATIONS AND ROADMAP

An actionable path forward

Proposed work, ordered by correctness and operating risk.

Phase	Deliverables	Exit evidence
A / Protect core flows	Server upload constraints; consistent Pro predicate; aligned platform enums; validated date parsing; payment atomicity and recovery; meaningful privacy terms.	Failure and concurrency tests; test-mode payment reconciliation; deletion/retention policy reviewed against actual data flows.
B / Establish reliability	Known job producer and provenance; freshness monitoring; tested cleanup policy; DB readiness; structured logs/metrics; backups and restore; environment validation.	Documented feed owner and run ledger; stale-feed alert; successful restore drill; health checks and operational runbook exercised.
C / Complete user journeys	Status editing and application deletion with ownership checks; usable study UI and seed process; clear plan restrictions and support links; terms and 404 routes.	Browser journey tests for Free, Pro, expired Pro and admin; API contract matches schema and UI.
D / Scale and discover ability	Queue heavy parsing/AI; shared rate-limit store; recommendation caching/query design; admin pagination; public-page metadata/prerendering; CI release gates.	Measured latency/cost, representative load tests, Search Console rendering/indexing evidence and deterministic release checks.

Current scale and accuracy limits

Recommendation quality depends on the external job corpus, canonical skill consistency and AI extraction. Pool caps bias scoring toward recent records. Simple skill overlap cannot capture role context, seniority nuance or transferable skills. Experience summation can overcount overlapping periods; date formats can fail. There is no calibration study, ranking evaluation dataset or fairness assessment in the repository.

Operational unknowns to resolve with the owner

Record the deployed revision, host/region, API origin, MongoDB backup and index state, provider capture settings, live plan prices, job producer and data-source permissions, actual retention policy and monitoring ownership. These details should become maintained configuration and runbook evidence, rather than assumptions.

Documentation maintenance

On each release: regenerate the route/model inventory, inspect changed controllers, re-run bounded checks and acceptance tests, recapture public screens, and update the source revision. Mark proposed work complete only when code and relevant runtime evidence support it.

17 / EVIDENCE REGISTER

Source register: backend and state

Click a path to open the exact reviewed revision on GitHub.

All R sources use immutable commit d72d00380934bf62b9e0e31eeaa24023ec461032. Source IDs printed in page footers provide a traceable route from statements and diagrams to implementation. Each file was fetched as text at that revision.

- **R01** backend/server.js
- **R02** backend/controllers/authController.js
- **R03** backend/middleware/authMiddleware.js
- **R04** backend/controllers/userController.js
- **R05** backend/services/geminiService.js
- **R06** backend/utils/calculateATSScore.js
- **R07** backend/services/recommendationService.js
- **R08** backend/controllers/jobController.js
- **R09** backend/controllers/applicationController.js
- **R10** backend/routes/jobRoutes.js
- **R11** backend/routes/payment.js
- **R12** backend/services/razorpayService.js
- **R13** backend/controllers/adminController.js
- **R14** backend/routes/adminRoutes.js
- **R15** backend/controllers/feedbackController.js
- **R16** backend/controllers/studyController.js
- **R17** backend/cron/cleanupJobsCron.js
- **R18** backend/middleware/multer.js
- **R19** backend/middleware/Ratelimiters.js
- **R20** backend/middleware/adminMiddleware.js
- **R21** backend/utils/plan.js
- **R22** backend/models/User.js
- **R23** backend/models/Resume.js
- **R24** backend/models/Job.js
- **R25** backend/models/Application.js
- **R26** backend/models/PaymentOrder.js
- **R27** backend/models/Feedback.js
- **R28** backend/models/SkillResource.js
- **R29** backend/models/PlatformAccount.js
- **R30** frontend/src/App.jsx
- **R31** frontend/src/main.jsx
- **R32** frontend/src/context/AuthContext.jsx
- **R33** frontend/src/context/ResumeContext.jsx

Source precedence
Routes determine whether a controller is reachable. Schemas determine persistence constraints. Controller expressions take precedence over outdated comments and README claims. Disabled imports or fields are not evidence of an active feature.

17 / EVIDENCE REGISTER

Source register: interface and routes

Public observations and primary provider references complement the code.

- **R34** frontend/src/pages/Resume.jsx
- **R35** frontend/src/pages/Recommendations.jsx
- **R36** frontend/src/components/JobCard.jsx
- **R37** frontend/src/pages/Applicatons.jsx
- **R38** frontend/src/pages/Dashboard.jsx
- **R39** frontend/src/pages/Profile.jsx
- **R40** frontend/src/pages/Pricing.jsx
- **R41** frontend/src/pages/Privacy.jsx
- **R42** frontend/src/pages/AboutMatchora.jsx
- **R43** frontend/src/pages/AdminDashboard.jsx
- **R44** frontend/src/pages/JobManagement.jsx
- **R45** frontend/index.html
- **R46** frontend/public/robots.txt
- **R47** frontend/public/sitemap.xml
- **R48** frontend/public/_redirects
- **R49** frontend/vite.config.js
- **R50** backend/package.json
- **R51** frontend/package.json
- **R52** backend/services/seedService.js
- **R53** backend/services/scrapers/naukriScraper.js
- **R54** backend/services/jobService.js
- **R55** backend/middleware/requirePro.js
- **R56** frontend/src/pages/Pro.jsx
- **R57** backend/utils/skillAliases.js
- **R58** backend/data/skillResources.js
- **R59** frontend/src/hooks/useInstallPrompt.js
- **R60** frontend/src/pages/Login.jsx
- **R61** backend/db/mongodb.js
- **R62** backend/utils/generateToken.js
- **R63** backend/routes/authRoutes.js
- **R64** backend/routes/userRoutes.js
- **R65** backend/routes/feedbackRoutes.js
- **R66** backend/routes/study.js

Live observations - 8 November 2026: L01 Home; L02 About; L03 Pricing; L04 Privacy. Screenshots are authentic public-page captures. No authenticated product data was used.

Primary guidance, reviewed 8 November 2026:
D01 Google Search Central - Ask Google to recrawl your URLs
D02 Google Search Central - JavaScript SEO basics
D03 Razorpay - Standard Checkout integration steps

17 / COVERAGE AND GLOSSARY

Review coverage and handover

A comprehensive source review with explicit runtime boundaries.

Repository area	Coverage and notable files
Root - 2 files	README.md and .gitignore. README checked against implementation; several claims are stale. No root .github/workflows directory or deployment manifest.
Backend - 46 files	43 JavaScript files, package.json, package-lock.json and .gitignore. All controllers, middleware, routes, models, services, cron, data, DB and utility modules retrieved and inspected.
Frontend - 49 files	43 text files plus 6 image/vector assets. All pages, components, contexts, hooks, CSS, HTML, Vite/Tailwind/PostCSS/ESLint config, public crawl/redirect files, README and lockfile retrieved or inventoried.
Image/vector asset inventory	public/icons.svg; public/pwa-192x192.png; public/pwa-512x512.png; src/assets/hero.png; src/assets/react.svg; src/assets/vite.svg. Binary/vector source assets were not visually audited; PDF screenshots come from the live browser.
Workflow / deployment scan	No Actions workflow files, Docker/Compose definitions, explicit platform manifests, .env.example or tracked test suite found. This is a statement about the pinned tree, not external hosting/account configuration.
Final branch check	main remained at d72d00380934bf62b9e0e31eeaa24023ec461032 when rechecked during document preparation. No upstream file, branch, issue or setting was modified.

Terms used in this document

Term	Meaning in Matchora
ATS score	Custom resume heuristic computed from AI-extracted structure; separate from job fit.
Skill score / match %	Rounded fraction of required job skill entries found in resume.skills.
Eligibility	Experience-month comparison used to exclude jobs.
Active Pro	Generally pro with future expiry; resume helper has a missing-expiry exception.
JobKey / jobId	Job.jobKey is copied into Application.jobId as a string.
Mounted	Connected to the Express server through an active router definition.
Observed / verified	Observed means seen on a public page; verified code means established by source, without implying deployed success.

Prepared as a read-only technical and product documentation deliverable. Maintain the evidence labels when reusing this material in product, engineering, security or stakeholder discussions.